

Can you collaborate?

Experiencing the Pain of Collaboration Without Version Control — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

Objective: In this activity, you must work in **groups of 5** to co-author two shared files: `file-1.txt` and `file-2.txt`. You need to submit a final story.

Section A — Rules of Engagement

Rule 1. Manual Collaboration Only

Git, GitHub, Google Docs, shared folders, and any other live collaboration tools are **not allowed**.

Rule 2. Share Files Through Messages Only

Files may only be exchanged through email, whatsapp or direct messages. No face-to-face discussions or file transfers are allowed.

Rule 3. Everyone Must Contribute

Every student must contribute at least **five meaningful sentences** and make changes to **both** `file-1.txt` and `file-2.txt` on their own laptop.

Rule 4. Merge and Submit

Your team must manually combine everyone's changes into the final versions of both files. One team member will submit the final files to the teacher.

Section B — The Collaborative Story Activity

Story Title: A Day Before the Final Exams

File 1: Preparing for the Exams

Write about:

- The last day before the exams.
- Students studying together.
- Funny moments in the classroom or library.
- Teachers giving last-minute advice.

File 2: Exam Day and After

Write about:

- Arriving at school on exam day.
- Experiences during the exam.

- Talking with friends after the exam.
- What everyone learned from the experience.

Student Assignments

Student	Contribution in File 1	Contribution in File 2
Student 1	Write the introduction and classroom atmosphere.	Write the beginning of exam day.
Student 2	Add how friends prepare together.	Add what happens during the exam.
Student 3	Add teachers' advice and last-minute revision.	Add students discussing the paper afterward.
Student 4	Improve the story with dialogues and descriptions.	Describe everyone's feelings after the exam.
Student 5	Edit and expand earlier sections, adding smooth transitions.	Write the conclusion and the lesson learned.

Final Instructions: Every student must edit both files on their own laptop. At the end, one team member should collect the latest versions of both files and submit them to the teacher.

Section C — Reflection on Manual Collaboration

Now that you have attempted this manual workflow, reflect on the process:

Q1. Is this process of **manually merging files** efficient? Why or why not?

Q2. Now given that all the files are merged, can you **track what part is written by whom**?

Q3. Can you think of a scenario in a real-world **software project** where the ability to "restore" a previous version would be critical?

Discussion: *In the world of software, we handle this using Version Control Systems. Large codebases and repositories often host projects with hundreds or even thousands of contributors working on the same code simultaneously. How do you think they manage this without overwriting each other's work?*

Section D — Understanding Git and GitHub

What is a Project? A project is a collection of files and folders that together make up something you're building — like a website, an app, a game, or even a collaborative story. When developers work on software, all the code, images, configuration files, and documentation are part of the project.

Q4. Think of an example of a project you might build. What **types of files** would it contain?

What is a Repository (Repo)? A repository is a special folder that contains your project *plus* the entire history of every change ever made to it. Git tracks who made each change, when they made it, and what exactly was changed. Think of it as a project folder with a built-in time machine.

Q5. How is a **repository** different from a regular folder on your computer?

What is Git? Git is a *version control system* — a tool that runs on your computer and tracks changes to your files. It lets you save snapshots of your project (called "commits"), go back to previous versions, and work on different features simultaneously without breaking things. Git works entirely on your local machine — you don't need the internet to use it.

Q6. Why is it useful to have a **history of all changes** made to a project?

What is GitHub? GitHub is a website that hosts Git repositories online. While Git works locally on your computer, GitHub lets you store a copy of your repository on the internet so you can access it from anywhere, share it with others, and collaborate with teammates. Think of it this way: **Git** is the tool, **GitHub** is the cloud service that stores your repositories.

Q7. If Git works offline on your computer, why would you also want to use **GitHub**?

Q8. Can you think of other services similar to GitHub? (Hint: there are alternatives for hosting Git repositories)

Section E — Setting Up Your Team Repository

If you do not already have a GitHub account, create one at <https://github.com>.

GitHub Username: _____

Only one member of your group should create a new **private repository** on GitHub:

1. Log in to GitHub and click **New Repository**.
2. Give the repository an appropriate name.
3. Select **Private** as the repository visibility.
4. Check "**Add a README file**", initializing the repository with a README file.
5. Create the repository.

Q9. Why do you think the repository is kept **private** instead of public?

The repository owner should now add collaborators:

1. Go to **Settings → Collaborators**.
2. Click **Add people** and invite group members using their GitHub usernames.

Each invited member should check their email or GitHub notifications and click **Accept Invitation**.

Q10. Why do collaborators need an **invitation** before they can modify a private repository?

What is Cloning? Cloning means creating a complete copy of a repository from GitHub (or another remote server) onto your local computer. When you clone, you get all the files *and* the entire history of the project. You can then work on the project offline and later sync your changes back to GitHub.

Each group member should now clone the repository. Open Git Bash (or Terminal) and run:

```
git clone <repository-url>
cd <repository-name>
ls
```

Q11. What files and folders do you currently see inside the repository?

Q12. After cloning, do you need internet access to make changes to your local copy?

Section F — Working with Git

Now let's start our team collaboration activity in this repository.

Only one member of the team should first delete the "README.md" file from this folder, and then create two new files in this folder - "file-1.txt" and "file-2.txt".

Before saving anything to Git, check the repository status:

```
git status
```

Q13. What does Git tell you about the deleted **README.md** file and the two new files?

Q14. Why do you think Git reports the deleted **README.md** file **differently** from the two new files?

Now stage the files and check the status again:

```
git add .
git status
```

Q15. What changed after running `git add`?

The Staging Area (Index): Git doesn't automatically include all your changes in a commit. Instead, it uses a **staging area** — a middle ground between your working directory and the repository. When you run `git add`, you're telling Git: "I want to include these specific changes in my next commit." This gives you fine-grained control over what gets saved. Think of it like packing a suitcase — you choose exactly what goes in before you close it.

Q16. Git now says the files are "**Changes to be committed.**" What do you think this means?

Q17. Why might it be useful to **stage only some files** instead of all changed files at once?

Q18. If you modified three files but only want to commit two of them, what command would you use instead of `git add .`?

What is a Commit? A commit is a snapshot of your project at a specific point in time. Each commit records what changed, who made the change, when it was made, and includes a message describing why. Commits are like save points in a video game — you can always go back to any previous commit if something goes wrong.

Now create your first commit:

```
git commit -m "Add story files"
```

Q19. Why is it useful to give every commit a **meaningful message**?

What is Push? Pushing means uploading your local commits to a remote repository (like GitHub). After you commit changes locally, they only exist on your computer. Running `git push` sends those commits to GitHub so others can see them and your work is backed up online.

Upload your changes to GitHub:

```
git push origin main
```

Q20. Visit your GitHub repository. Can you now see your files on **GitHub**?

Section G — Collaborating as a Team

Important: Do this one-by-one. At any point of time, one member from the team will be working and others will be waiting. **Only one student should edit the project at a time.** Only one student must follow this workflow before the next student begins. After the first student complete, he / she should stop and then second student should begin.

What is Pull? Pulling means downloading the latest changes from a remote repository (like GitHub) to your local computer. Before you start working, you should always pull to make sure you have everyone else's latest changes. This prevents conflicts where two people edit the same thing differently.

Before making any changes, always download the latest version first:

```
git pull origin main
```

Q21. Why is it important to **pull** before editing?

Now, student should add his / her assigned contribution to both story files, save it and then check what Git detected:

```
git status
```

Q22. What files has Git detected as **modified**?

Stage your changes, commit with your name, and push:

```
git add .  
git commit -m "Add [Your Name]'s contribution"  
git push origin main
```

Once your push is successful, inform the next teammate. They should **not start editing** until they have first run `git pull origin main`.

Repeat this workflow until every member of the group has contributed.

After everyone has contributed, explore the repository history:

```
git log
```

Visit the GitHub repository and answer the following:

Q23. What information is stored for **each commit**?

Q24. Can you identify **who contributed each change**?

Q25. How is this workflow **different** from manually sending files through email or messages?

Q26. Looking back to the activity, think about how does **Github** help when multiple people are involved?
